1.5 Parallel Computing

Goal: Define PDC vocabulary including **processor**, **CPU**, **core**, **sequential computing**, and **parallel computing.** Give some examples of parallel and distributed computing.

The section titled "Computer Tour" introduced the basic hardware components in a computer system, including the processor, where the actual computations are performed. The processor is also referred to as the **Central Processing Unit** or **CPU**. The main electronic component inside the CPU is the transistor. In general, the more transistors that can be put into a chip the more powerful the computer. However, more power requires more energy and produces more heat that must be dissipated. One of the solutions for increasing performance is to put multiple processors into the CPU. When a CPU contains multiple processors, they are often called "**cores**" in the specifications for the chip, as in "the Intel Core i7 processor has 8 cores." However, "core" and "processor" are also often used interchangeably, so this type of computer system may also be termed a **multiprocessor.** Having more than one core allows programs to be divided among these individual processors. This arrangement allows tasks to run at the same time on different cores and is one form of **parallel computing**. **Parallel computing** is a computational model where tasks are broken up into smaller pieces that can each be run on a different processor at the same time (in parallel). These tasks might be entire programs or they might be a program that has been broken down into smaller pieces.

Table 1.5.1: Sequential vs. Parallel Computing

Sequential Computing	**Parallel Computing**
Sequential computing is the standard programming model where the CPU executes each operation of the program in order, one at a time on the same processor.	Parallel Computing is a computational model that breaks programs into smaller sequential operations and performs those operations at the same time, in parallel on different processors.

Often in computing the same computation needs to be done over and over. **Data parallelism** is when a dataset can be divided and a computational task performed on each section separately by different processors. Consider a program that will add together 1000 numbers. If there are 10 processors and they each add 100 numbers to find the sum at the same time this operation will take much less time than one processor adding all 1000 numbers. Of course the 10 individual sums also must be added together. This part of the process is called the **synchronization** phase. Consider this same example if we had 1000 processors. It would not be advantageous to split the task between 1000 processors because the synchronization phase would likely take at least as long as having one processor do the task sequentially. In addition, if there are 10 processors, but the task is divided unevenly, for example if processor 1 is given 500 of the numbers and the rest of the numbers are distributed among the remaining 9, the processors, the entire task will be waiting for processor 1 to finish. This is an example of **load imbalance**, when one process

A program is generally composed of many individual tasks. Sometimes it is possible for these tasks to be performed at the same time. In this case, it is possible to implement a parallel processing solution known as **task parallelism,** where each processor can handle individual tasks. Note that the depends on whether or not it the program can be broken up into different tasks. A load imbalance can also occur in this case if one of the processors has more processing to do than other processors.

Further reading:

Adams J.C., Brown R., Matthews S.J., Shoop E., and others. *PDC for Beginners*, 0.1 Edition. Available online: https://dx.doi.org/10.55682/VXWY1300

Figure 1.5.1: Data parallelism is when data is divided among processors to perform the same task to each subset.

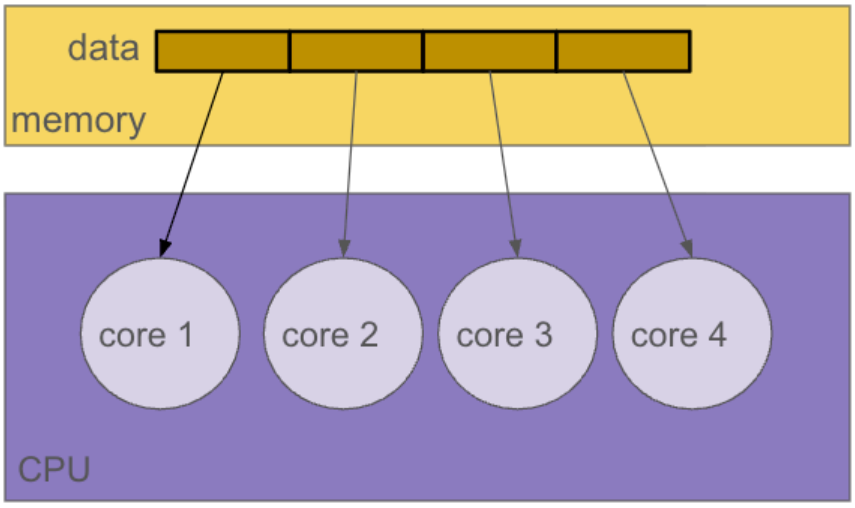
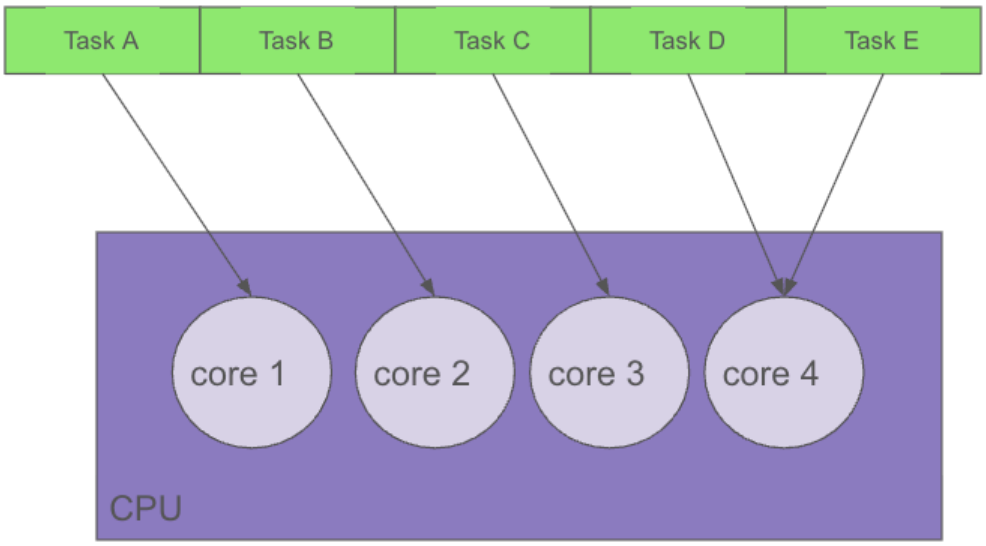


Figure 1.5.2: Task parallelism



PARTICIPATION ACTIVITY | 1.5.1: Parallel vs. Sequential

1) _____ is a computational model where tasks are broken up into smaller pieces that can each be run on a different processor at the same time.

- Sequential Processing
- Synchronized Processing
- Parallel Processing
- Double-Time Processing

2) An image processing task involves doing the same operation on blocks of pixels in the image. This process would be an ideal candidate for _____.

- Data parallelization
- Sequential Parallelization
- Distributed Parallelization
- Central Processing Unit